HRIS

Human Resource Information System

Senior Developer Handover & Architecture Guide

Stack	Architecture	Auth
ASP.NET Core 10 + React + PostgreSQL	Decoupled REST API + SPA	JWT Bearer + ASP.NET Identity + Claims

This document is intended for incoming senior developers and serves as the primary reference for understanding the codebase architecture, file connections, and debugging starting points.

1. System Specifications & Tech Stack

The following specifications define what every senior developer must have installed and understand before working on this codebase. These are hard requirements — the system will not build or run without them.

1.1 Backend Requirements

Dependency	Version	Purpose & Notes
.NET SDK	10.0 (required)	Runtime and build toolchain for ASP.NET Core. Download at dotnet.microsoft.com
ASP.NET Core	10.0 Web API	Backend framework. Handles HTTP routing, middleware, and dependency injection.
Entity Framework Core	10.0	ORM for database access. Manages migrations and schema via C# model classes.
Npgsql EF Core Provider	Latest compatible	PostgreSQL driver for EF Core. NuGet: Npgsql.EntityFrameworkCore.PostgreSQL
ASP.NET Core Identity	Included in .NET 10	User management, password hashing, and role/claim storage.
SignalR	Included in .NET 10	WebSocket hub for real-time push notifications to connected clients.
DotNetEnv	Latest	Loads .env files into environment variables at startup. Install via NuGet.
System.Text.Json	Included in .NET 10	JSON serialization. Configured for dynamic DTOs and EF Core JSONB column mapping.
dotnet-ef (CLI tool)	Latest global	Required for running migrations. Install: dotnet tool install --global dotnet-ef

1.2 Frontend Requirements

Dependency	Version	Purpose & Notes
Node.js	18+ LTS	JavaScript runtime for Vite and npm. Download at nodejs.org.
npm / yarn	Bundled with Node	Package manager. Run npm install inside /hris before starting the dev server.
Vite	Latest (package.json)	Build tool and dev server. Starts with npm run dev. Replaces Webpack.
React	18+	UI framework. All components are functional with hooks.
TypeScript	5+	Strongly typed JS. All source files use .ts / .tsx extensions.
Tailwind CSS	3+	Utility-first CSS. Config in tailwind.config.ts.
React Admin (ra-core)	Latest	Data framework. Drives List, DataTable, forms, and the central dataProvider.

1.3 Infrastructure & Ports

Requirement	Detail
PostgreSQL 14+	Must be running before backend starts. Configure in DB_CONNECTION_STRING inside backend/.env.
SMTP Server	Required for email features (password reset, notifications). Set SMTP_HOST, SMTP_USER, SMTP_PASS in .env.
Vercel (frontend)	Frontend is deployed to Vercel. vercel.json in root handles SPA routing and push-state config.
Port 5107	Backend local port. Must be free before starting. setup-backend.ps1 kills any process using it automatically.
Port 5173 / 5174	Vite dev server defaults. Both are whitelisted in the backend CORS policy in Program.cs.
Port 3000	Alternative frontend port. Also whitelisted in CORS.

INSTALL ORDER	1. Install .NET 10 SDK -> 2. Install Node.js 18+ -> 3. Start PostgreSQL -> 4. Configure backend/.env -> 5. Run setup-backend.ps1 -> 6. Run npm install then npm run dev inside /hris

1.4 Environment Variables (backend/.env)

The backend reads all secrets from backend/.env at startup via DotNetEnv. Copy .env.example to .env and fill in every value before running the server.

Variable	Purpose
`DB_CONNECTION_STRING`	Full PostgreSQL connection string (Host, Port, Database, Username, Password)
`JWT_SECRET`	Signing key for JWT tokens. Must be at least 32 characters long.
`JWT_ISSUER`	Token issuer identifier (e.g. hris-api)
`JWT_AUDIENCE`	Token audience identifier (e.g. hris-client)
`ENABLE_SWAGGER`	Set true to expose /swagger in development. Always false in production.
`APPROVER_EMAIL`	Email for the seeded Approver test account (created on first migration)
`VIEWER_EMAIL`	Email for the seeded Viewer test account (created on first migration)
`USER_SAMPLE_EMAIL`	Email for the seeded standard Employee test account
`SMTP_HOST / SMTP_USER / SMTP_PASS`	Credentials for the SmtpEmailSender service used by Identity email features

2. High-Level Architecture

The HRIS is a fully decoupled Client-Server system. The frontend and backend are separate applications that communicate exclusively via HTTP REST APIs and WebSocket (SignalR) connections.

Layer	Technology	Location / Notes
Web Client	React (TypeScript + Vite) + Tailwind CSS	/hris — SPA served via Vercel
REST API	ASP.NET Core 10 Web API	/backend — runs on port 5107 locally
Database	PostgreSQL via EF Core (Npgsql)	Managed by EF Core migrations
Real-Time	SignalR WebSockets	Hub at /hubs/notifications
Auth	JWT Bearer + ASP.NET Identity	Claims-based RBAC

1.1 Request Execution Pipeline

Every HTTP request from the frontend follows this exact flow:

```
React Frontend (dataProvider.ts)
    |-- Attaches Bearer token from localStorage
    v
Program.cs / Middleware (JWT Auth, CORS)
    v
Controller (receives DTOs, returns ApiResponse<T>)
    v
Service Layer (business logic, permission checks)
    v
ApplicationDbContext --> PostgreSQL
```

NOTE	Controllers never touch the database directly. All DB access goes through Services, which use ApplicationDbContext.

2. Backend Engine (/backend)

The backend follows a strict MVC pattern. It is an ASP.NET Core 10 Web API with no server-rendered views — purely a REST API consumed by the React frontend.

2.1 Core Files & Their Roles

File / Folder	Purpose	Key Connections
Program.cs	App bootstrapper. Registers all services, middleware, JWT, CORS, SignalR.	Entry point. Connects everything.

appsettings.json + .env	Configuration. DB connection string, JWT secret, email credentials.	Loaded by DotNetEnv at startup.
Data/ApplicationDbContext.cs	EF Core database context. Maps C# models to PostgreSQL tables.	Used by all Services.
Data/SeedDatabase.cs	Seeds default roles, SuperAdmin, Approver, Viewer accounts on first run.	Called from Program.cs at startup.
Models/*.cs	C# classes representing DB tables (ApplicationUser, AccomplishmentReport, etc.).	Consumed by ApplicationDbContext.
DTOs/*.cs	Data Transfer Objects defining exact JSON shapes for API requests/responses.	Used by Controllers.
Services/*.cs	Business logic layer. All DB queries and permission checks live here.	Called by Controllers.
Controllers/*.cs	HTTP endpoints. Validates input, calls Services, returns ApiResponse<T>.	Sits between DTOs and Services.
Hubs/NotificationHub.cs	SignalR WebSocket hub for real-time push notifications.	Connected to NotificationsController & frontend.
Migrations/	EF Core migration history. DO NOT DELETE.	Required for dotnet ef database update.

2.2 Controllers & Responsibilities

Controller	Route	Responsibility
AuthController.cs	POST /api/auth/login	Issues JWT tokens, handles login/logout.
EmployeesController.cs	GET/POST/PUT /api/employees	Full employee CRUD, role & claim management.
AccomplishmentReportsController.cs	GET/POST/PUT/DELETE /api/accomplishmentreports	AR submission, editing, deletion for report owners.
ArReviewsController.cs	GET /api/ar-reviews	AR review queue and individual report view for Approvers/Viewers/Admins.
DashboardController.cs	GET /api/dashboard	Aggregated statistics for frontend charts.
NotificationsController.cs	GET/PUT /api/notifications	Fetch and mark-read for historical notifications.

2.3 Unified API Response Format

All endpoints return the same wrapper structure. The frontend dataProvider.ts is built to unwrap this automatically:

```
{
  "success": true,
  "data": { ... },
  "message": "Operation successful"
}
```

3. Authentication & Access Control (RBAC)

Authentication uses JWT Bearer tokens. Authorization uses a combination of ASP.NET Identity Roles and custom Claims for granular permissions.

3.1 Roles vs Claims

Type	Values & Behavior
Role: SuperAdmin	Full system access. Cannot be downgraded via API. Sees all data across all departments.
Role: HR Management	Admin access. Sees all reports, can approve/return any AR.
Role: Employee	Base role for all standard users. Default on account creation.
Claim: Permission = Approver	Can approve or return Accomplishment Reports assigned to them. Sees AR Review queue.
Claim: Permission = Viewer	Read-only access to AR Review queue. Cannot approve or return. Department-scoped.

CRITICAL Claims are stored in AspNetUserClaims (DB), NOT in the JWT token by default. If a user's claim is updated, they must log out and log back in for the new token to reflect the change.

3.2 Policy Definitions (Program.cs)

Two named authorization policies control AR access:

Policy Name	Who Passes
CanViewARReview	SuperAdmin, HR Management, Approver claim, Viewer claim
CanApproveAR	SuperAdmin, HR Management, Approver claim ONLY (Viewer is excluded)

DEBUG TIP If an Approver gets 403 on AR Review, the claim is likely missing from their JWT. Add a debug endpoint: [HttpGet("debug-claims")] that returns User.Claims to verify what the token contains.

4. Frontend Client (/hris)

A React SPA (TypeScript + Vite) using React Admin as the data management framework. All API communication goes through a central dataProvider.ts file.

4.1 Key Frontend Files

File	Purpose
src/dataProvider.ts	Central HTTP client. Attaches JWT token, unwraps ApiResponse<T>, maps reportId -> id for React Admin.
src/authProvider.ts	Handles login, logout, token storage in localStorage, and permission checks for the UI.
src/auth/roles.ts	Constants for role/claim strings used across components (ROLE_EMPLOYEE, etc.).
vite.config.ts	Build configuration. Defines API proxy rules for local development.
tailwind.config.ts	Design system tokens (colors, spacing, breakpoints).

4.2 How dataProvider.ts Works

React Admin calls dataProvider methods (getList, getOne, create, update, delete). Each method builds the correct API URL, attaches the Bearer token, and handles the ApiResponse<T> unwrapping:

```
// Token is attached from localStorage:
headers.set('Authorization', `Bearer ${token}`);

// ApiResponse<T> is automatically unwrapped:
if (json.success && json.data) return json.data;

// update() auto-routes status changes to /status endpoint:
const path = data.status ? `/{id}/status` : `/{id}`;
```

5. Database Schema

The database is managed exclusively by EF Core. Never modify the schema with raw SQL — always use migrations. The connection string is defined in backend/.env as DB_CONNECTION_STRING.

5.1 AccomplishmentReports Table

Core table storing Accomplishment Reports. Tasks and feedback are stored as JSONB arrays, reducing row-per-task complexity.

Column	Type	Description

ReportId	int (PK)	Primary key, auto-increment
Date	date	Report date (Required)
Title	string (50)	Report title
Status	string	Pending | Draft | Approved | Returned | Returned_Draft
CreatedAt	timestamp	UTC creation time (auto)
UserId	string (FK)	Owner/Submitter — maps to AspNetUsers.Id
ReceiverId	string (FK)	Assigned Approver — maps to AspNetUsers.Id
ViewerId	string (FK)	Assigned Viewer (CC) — maps to AspNetUsers.Id
ReturnedById	string	ID of user who last returned the report
ReturnFeedback	string	Feedback message when report is returned
Tasks	Jsonb (client: 50 | tasks: 500)	List of ARTask: {Client, TaskName, Particulars, StartTime, EndTime}
FeedbackHistory	jsonb	List of ARFeedback: {Message, AuthorId, AuthorName, Timestamp, Action}
ViewedAt	timestamp	When assigned Viewer first opened the report
ViewedById	string	Viewer user ID
ViewedByName	string	Viewer full name

5.2 Other Key Tables

Table	Purpose
AspNetUsers (extended)	Standard Identity users + FullName, Position, Department, EmployeeID custom columns.
AspNetRoles / AspNetUserRoles	Auto-generated. Stores SuperAdmin, HR Management, Employee roles.
AspNetUserClaims	Stores Permission=Approver and Permission=Viewer claims per user.
Notifications	Stores real-time alerts: UserId, Title, Body, LinkTo, EventType, IsRead, CreatedAt.
DeletedAccomplishmentReports	Audit archive of deleted reports. Preserves all original data + who deleted and when.

5.3 Making Schema Changes

Always follow this sequence when modifying the database:

1. Edit the C# model class inside backend/Models/*.cs
2. Generate a new migration:
```
dotnet ef migrations add DescriptiveNameOfChange
```
3. Apply to local database:
```
dotnet ef database update
```
4. Commit both the model change AND the new Migrations/ files together.

WARNING	Never delete the Migrations/ folder. EF Core needs the full migration history to build the schema on a fresh machine. See backend/MERGING_GUIDE(FOR NEW TABLE DB).md for git merge conflicts in migrations.

6. Real-Time Notifications (SignalR)

The system uses SignalR WebSockets to push live notifications without HTTP polling.

Item	Detail
Hub Class	Hubs/NotificationHub.cs
Hub Route	/hubs/notifications (configured in Program.cs)
User Groups	Each user joins group user-{userId} on connection
Client Event	ReceiveNotification — frontend listens for this
Triggers	Report Submitted, Approved, Returned, Viewed
Persistence	Every notification is also saved to the Notifications table in PostgreSQL

7. Environment Setup (First-Time)

7.1 Prerequisites

- .NET 10 SDK — https://dotnet.microsoft.com/download
- PostgreSQL instance (local or remote)
- Node.js 18+ (for frontend)
- dotnet-ef global tool (installed by setup script)

7.2 Backend Setup

5. Navigate to the backend/ folder
6. Copy .env.example to .env and configure:

```
DB_CONNECTION_STRING=Host=localhost;Port=5432;Database=hris_dev;Username=postgres;Pass
word=yourpassword
JWT_SECRET=your_secure_key_at_least_32_chars_long
JWT_ISSUER=hris-api
JWT_AUDIENCE=hris-client
ENABLE_SWAGGER=true
```

7. Run the automated setup script (recommended):

```
cd backend\setup_backend
.\setup-backend.ps1
```

The script will: stop any lingering backend processes, restore NuGet packages, run EF migrations, and start the server on port 5107.

8. OR run manually:

```
dotnet restore
dotnet ef database update
dotnet run
```

7.3 Default Seeded Accounts

After the first successful migration, SeedDatabase.cs auto-creates these test accounts:

Account	Role / Claims
admin@hris.local	SuperAdmin — full system access
Defined in .env (APPROVER_EMAIL)	Employee role + Permission=Approver claim
Defined in .env (VIEWER_EMAIL)	Employee role + Permission=Viewer claim
Defined in .env (USER_SAMPLE_EMAIL)	Employee role — standard user

8. Debugging Common Issues

8.1 403 Forbidden on AR Review Endpoints

Most common cause: the user's claims are not in their JWT token.

9. Add this temporary debug endpoint to ArReviewsController.cs:

```
[HttpGet("debug-claims")]
public IActionResult DebugClaims() =>
    Ok(User.Claims.Select(c => new { c.Type, c.Value }));
```

10. Hit GET /api/ar-reviews/debug-claims while logged in as the affected user.
11. If Permission/Approver is missing from the response, the user must log out and log back in to get a fresh token after their claim was assigned.
12. If the claim is present but still 403, check the policy definition in Program.cs.

8.2 AR List Shows Wrong Reports

The GetAllReportsAsync method in AccomplishmentReportService.cs branches by role and claim:

- SuperAdmin / HR Management: sees ALL non-draft reports
- Approver or Viewer claim: sees only their OWN reports (My ARs)

- Standard Employee: sees only their own reports

The AR Review queue (GetReviewQueueAsync) uses separate logic — Approver and Viewer claims see all reports EXCEPT their own.

8.3 EF Core / Database Errors

- dotnet ef database update fails: Check DB_CONNECTION_STRING in .env is correct and PostgreSQL is running.
- Migration conflict after git merge: Read backend/MERGING_GUIDE(FOR NEW TABLE DB).md carefully before resolving.
- 'relation does not exist' error: The migration has not been applied. Run dotnet ef database update.

8.4 SignalR Not Receiving Events

- Ensure the frontend WebSocket URL matches the hub route: /hubs/notifications
- Check that the user is authenticated — SignalR connections require the Bearer token.
- Verify the NotificationService is being called after the triggering action (AR submitted, approved, etc.).

GENERAL DEBUG TIP	Run dotnet watch run for the backend during development. EF Core logs every SQL query to the console — this is the fastest way to trace permission failures and unexpected query results.

9. Quick File Reference

File	Go here when...
backend/Program.cs	Changing middleware, CORS, JWT config, or auth policies
backend/Data/ApplicationDbContext.cs	Adding new DB tables or relationships
backend/Data/SeedDatabase.cs	Changing default seeded accounts or roles
backend/Services/AccomplishmentReportService.cs	Debugging who sees which reports; approval logic
backend/Services/EmployeeService.cs	Role/claim assignment logic for employees
backend/Controllers/ArReviewsController.cs	AR Review endpoint access control issues
backend/Controllers/AccomplishmentReportsController.cs	AR submission, edit, delete endpoint issues
backend/Hubs/NotificationHub.cs	Real-time notification connection/group issues
hris/src/dataProvider.ts	API call failures, token not attached, response not parsed
hris/src/authProvider.ts	Login/logout issues, permission checks in UI
backend/Migrations/	DB schema history — never delete this folder

`backend/setup_backend/setup-backend.ps1`	Automated environment setup for new developers

When in doubt, read the service layer first. Business logic, permission checks, and query filtering all live in Services/.cs.*